Transferência de arquivos Peer to Peer

Trabalho Prático 2 de Sistemas Distribuídos

CEFET-MG | Professora Michelle Hanne | 2026/2

Alunas: Thainá Martins e Giovana Laura

Código-fonte: https://github.com/ThainaCMartins/TP2-Sistemas-Distribuidos

1 Objetivo e resultado

O projeto implementa um sistema P2P em que cada processo pode receber e servir blocos do mesmo arquivo. A execução automatizada de 27 de outubro de 2026 concluiu as 24 combinações de teste e validou 48 cópias por tamanho e SHA-256. Testes adicionais demonstraram compartilhamento antes da conclusão, troca bidirecional e retomada com revalidação de dados locais.

2 Organização dos peers

A solução usa Python e sua biblioteca padrão. Um servidor TCP iniciado por asyncio.start_server atende conexões de entrada, enquanto tarefas criadas com asyncio.open_connection solicitam blocos a cada vizinho configurado. Todas as tarefas compartilham o mesmo estado de armazenamento no processo. O argumento --seed-file indica que um nó começa com o arquivo completo; os demais executam o mesmo programa sem esse argumento.

Cada JSON de configuração contém identificador, IPv4, porta e uma lista estática de vizinhos. A implementação dispensa tracker, permitido como opcional no enunciado. Os peers recebem previamente o mesmo metadado; o protocolo de rede transporta disponibilidade e blocos. Cada processo transfere um arquivo e permanece atendendo pedidos após concluir, até ser encerrado.

3 Fragmentação e metadados

O arquivo de origem é lido em blocos nominais de 1024 ou 4096 bytes. O último pode ser menor. O metadado JSON registra versão, nome, tamanho exato, tamanho de bloco, SHA-256 de cada bloco, SHA-256 global e identificador do conjunto. Esse identificador é o SHA-256 dos outros campos em JSON canônico com chaves ordenadas, impedindo a mistura acidental de conjuntos incompatíveis.

Os módulos separam responsabilidades: protocol.py define os quadros TCP; storage.py gerencia metadados e arquivos; peer.py coordena conexões e downloads. Os scripts de experimento iniciam os processos, verificam as cópias e preservam os resultados e logs.

4 Protocolo e concorrência

O TCP fornece um fluxo de bytes. Por isso cada quadro começa com quatro bytes contendo o comprimento do cabeçalho JSON em ordem de rede. Seguem o JSON UTF-8 e a quantidade de bytes binários declarada em payload_len. readexactly trata quadros divididos ou agrupados pelo TCP; drain controla o fluxo de envio. O cabeçalho é limitado a 64 KiB e o conteúdo binário a 4 MiB.

Pedido	Resposta	Conteúdo principal
HELLO	HELLO_ACK	Identificação do peer e do arquivo
HAVE	BITFIELD	Quantidade de blocos e bitmap binário
GET	BLOCK	Índice e bytes do bloco solicitado
GET inválido	ERROR	Índice e motivo da indisponibilidade

Após o handshake, cada tarefa consulta a disponibilidade de seu vizinho e embaralha os blocos ainda ausentes. As conexões são persistentes. Um conjunto inflight reserva índices durante as solicitações e evita pedidos simultâneos do mesmo bloco. A disponibilidade é atualizada após até 64 blocos aceitos; consultas sem trabalho útil aguardam 50 ms. Falhas de conexão ou protocolo geram registro e nova tentativa após 150 ms.

5 Recebimento e compartilhamento

Um bytearray registra os blocos locais válidos. O receptor confere índice, tamanho esperado e SHA-256 antes de escrever os dados no deslocamento correspondente do arquivo parcial. Somente após a gravação o índice é marcado como disponível. Não há suspensão da tarefa entre a escrita e a marcação; o servidor pode compartilhar esse bloco na consulta seguinte, mesmo que outros ainda faltem.

A remontagem ocorre pela escrita dos blocos em suas posições, aceitando chegada fora de ordem. Ao completar o bitmap, o programa sincroniza a gravação, compara o tamanho e o SHA-256 global e publica o nome final por renomeação. Não precisa manter todo o conteúdo em memória. O servidor continua ativo depois da publicação.

6 Falhas e recuperação

No reinício, o programa revalida os blocos do arquivo parcial. Apenas os íntegros são anunciados e reutilizados; dados danificados são baixados novamente. O download tem prazo global configurável, com padrão de 120 segundos. Se não houver fonte acessível para um bloco necessário, o programa encerra com erro ao atingir esse prazo. A persistência de cada bloco individual após queda de energia não é garantida; a revalidação protege a integridade na retomada.

7 Método de avaliação

Os estudos usam Linux 6.18.55 x86_64 e Python 3.12.14, em um único host, com TCP IPv4 em loopback. Cada peer é um processo independente. A matriz usa vizinhança estática em malha completa e atraso artificial zero. Para cada tamanho foi criado um arquivo aleatório com os.urandom, reutilizado nas quatro combinações correspondentes. Os experimentos iniciaram um seeder A e um ou três leechers.

Foram cruzados os seis tamanhos do enunciado, duas quantidades totais de peers e dois tamanhos de bloco, com uma execução por combinação. Adotou-se base binária: KiB significa 1024 bytes e MiB significa 1048576 bytes. O tempo apresentado é o maior intervalo entre READY e COMPLETE dos leechers de cada cenário; inclui obtenção e validação dos dados, mas não a inicialização do interpretador. O CSV também guarda o intervalo externo de lançamento até verificação.

Tabela 1 Tempos observados em segundos

Arquivo	2 peers 1024 B	2 peers 4096 B	4 peers 1024 B	4 peers 4096 B
10 KiB	0,023	0,165	0,023	0,064
20 KiB	0,164	0,025	0,165	0,026
1 MiB	0,227	0,207	0,186	0,212
5 MiB	1,111	0,412	0,934	0,272
10 MiB	2,267	0,502	1,978	0,610
20 MiB	5,706	1,109	4,792	1,007

Todos os 24 cenários terminaram com sucesso. As 48 cópias dos leechers tiveram tamanho e SHA-256 iguais aos respectivos originais em uma verificação externa ao programa de transferência. A auditoria conferiu 184472 recebimentos, suas fontes estáticas e os envios correspondentes. Não houve eventos FAILED nem BLOCK_REJECTED na matriz. Os registros incluem 75564 envios feitos por leechers, dos quais 74420 ocorreram enquanto o emissor ainda estava incompleto.

As evidências ficam em evidencias/matriz: resultados.csv e resultados.json contêm medidas por cenário; ambiente.json descreve a execução; cada pasta contém metadados, configurações, logs JSONL e completion.json por peer. Os binários da matriz foram removidos somente depois da comparação de integridade, para reduzir o pacote. A opção --manter-arquivos permite preservá-los em uma nova execução.

8 Verificações funcionais

Além da matriz, foram executados 13 testes automatizados de protocolo e armazenamento e três experimentos com processos independentes. Os testes cobrem quadros TCP fragmentados, agrupados e truncados, comprimento inválido, timeout, bitmap, recebimento fora de ordem, último bloco menor, duplicação, rejeição de conteúdo corrompido, retomada, arquivo vazio e metadados inconsistentes.

Compartilhamento durante o download

Na topologia estática A-B-C-D, C não possui A como vizinho e D conhece somente C. O original tinha 1048613 bytes, com 1025 blocos de 1024 bytes nominais. C recebeu os 1025 blocos de B e D recebeu os 1025 de C. Os logs registraram 1012 envios de B e 1012 de C enquanto cada um ainda estava incompleto. As três cópias finais foram verificadas. O atraso de 1 ms por bloco servido por A tornou esse comportamento observável.

A evidência de um bloco servido inclui peer, destinatário, índice, bytes e while_incomplete. Esse último campo foi registrado como verdadeiro antes da conclusão do emissor. A impossibilidade de C e D obterem os blocos diretamente de A também confirma o encaminhamento por peers intermediários.

Comunicação bidirecional

Dois peers começaram com metades complementares dos blocos válidos, sem seeder completo inicial. A recebeu 512 blocos de B; B recebeu 513 de A. Ambos também serviram blocos enquanto incompletos: 407 envios em A e 512 em B. Ao final, as duas cópias correspondiam ao original. O experimento comprova que cliente e servidor operam no mesmo processo e participam da mesma transferência.

Retomada e revalidação local

O controlador interrompeu B após o registro de 31 blocos recebidos e alterou um byte do primeiro bloco registrado, de índice 87. No reinício, B reconheceu 30 blocos válidos e voltou a solicitar o danificado. O índice 87 aparece duas vezes entre os recebimentos, antes e depois da alteração. O arquivo final voltou a corresponder ao original por SHA-256 e tamanho, demonstrando que o programa aproveita dados corretos e não confia apenas na existência do parcial.

Reprodução das verificações

A partir da pasta do projeto, execute python3 -m unittest discover -v para os testes básicos e python3 validar_funcional.py --out validacao_local para os três experimentos. As topologias, logs e resumos desta execução estão em evidencias/funcional. O guia LEIA_PRIMEIRO.md apresenta também a execução manual em dois ou quatro terminais.

9 Análise dos resultados

O tamanho do bloco altera a quantidade de pedidos, verificações e operações registradas. Para 20 MiB, blocos de 1024 bytes geram 20480 partes; blocos de 4096 bytes geram 5120. Na execução com dois peers, o maior tempo de download passou de 5,706 s para 1,109 s. Com quatro peers, passou de 4,792 s para 1,007 s. A redução é compatível com menor quantidade de operações por arquivo nesse ambiente, sem isolar individualmente o custo de rede, disco, hash e logs.

Com quatro peers, três leechers completam o arquivo e também distribuem blocos entre si. Para 20 MiB e blocos de 1024 bytes, o maior intervalo observado foi menor que no caso de dois peers. Isso não estabelece que adicionar peers sempre reduz o tempo: todos os processos compartilham CPU, disco e loopback, e o estudo possui uma observação por combinação. Não foram estimados intervalos de confiança ou ganho em rede física.

Nos arquivos de 10 e 20 KiB, a janela de 20 ms usada para verificar a conclusão e possíveis reconexões após 150 ms têm peso comparável à transferência. Por isso pequenas diferenças e picos nesses casos não permitem concluir sobre o desempenho do modelo. A matriz deve ser lida como avaliação funcional acompanhada de medidas descritivas.

10 Escopo e conclusão

A solução cobre os requisitos de simetria, vizinhos estáticos, fragmentação, solicitação concorrente, compartilhamento parcial, remontagem e integridade. Os testes mostram conclusão correta em todas as combinações exigidas e recuperação de blocos válidos após interrupção. Não há tracker, descoberta automática, obtenção remota do metadado, autenticação ou criptografia de transporte. Cada processo gerencia um arquivo. As operações locais de disco e hash são síncronas, embora os sockets sejam não bloqueantes, o que pode limitar a aplicação em outros volumes ou ambientes.

A reprodução completa usa python3 executar_testes.py --out resultados_local --manter-arquivos. O controlador valida cada cópia, registra as fontes e mantém arquivos e logs. Alterações de ambiente ou novas repetições devem ser registradas junto aos resultados correspondentes.

Referências

HANNE, Michelle. Sistemas Distribuídos. Trabalho Prático 2: Transferência de Arquivos Peer-to-Peer. CEFET-MG, 2026/2. Enunciado, 2 páginas.

PYTHON SOFTWARE FOUNDATION. Streams. Disponível em: https://docs.python.org/3/library/asyncio-stream.html.

PYTHON SOFTWARE FOUNDATION. hashlib. Disponível em: https://docs.python.org/3/library/hashlib.html.

PYTHON SOFTWARE FOUNDATION. struct. Disponível em: https://docs.python.org/3/library/struct.html. Documentação consultada em 27 out. 2026.